An Experimental Comparison of Sorting Algorithms: Performance Across Input Types and Scales

Maksat Kerimov

Department of Computer Science,

West University Timişoara

maksat.kerimov10@e-uvt.ro

May 2026

Abstract

This paper presents an experimental comparison of ten sorting algorithms: Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Merge Sort, Quicksort, Heap Sort, TimSort, Counting Sort, and Radix Sort. We measure actual running time and operation counts across five input sizes (from 1 000 to 500 000 elements) and four input distributions (random, already sorted, reverse sorted, and nearly sorted). The results confirm the expected $O(n \log n)$ advantage of the comparison-based advanced algorithms but also reveal a number of surprises: Insertion Sort outperforms Merge Sort on nearly-sorted data up to roughly 10 000 elements, and Quicksort is consistently the fastest comparison-based sort in practice despite its $O(n^2)$ worst case. The non-comparison sorts (Counting Sort, Radix Sort) dominate on integer inputs but become inapplicable for general data. All benchmark code and raw results are available at https://github.com/maksatkerimov10-sys/MPI-paper.

1 Introduction

Motivation

Sorting is arguably the most studied problem in computer science. Almost every non-trivial program sorts data at some point, and a large fraction of CPU time in real systems is spent inside sorting routines. Despite this, students (myself included, before doing this work) often leave algorithms courses with a somewhat simplified picture: "use Merge Sort or Quicksort, avoid Bubble Sort". The full story is more interesting.

Different algorithms have genuinely different strengths. A method that is terrible for random data can be excellent for nearly-sorted input. An algorithm with optimal asymptotic complexity can be slower in practice than one with worse bounds, simply due to cache behaviour and constant factors. The goal of this paper is to go beyond big-O notation and look at what actually happens when you run these algorithms on a real machine.

Informal Description of the Approach

We implemented ten sorting algorithms in Python 3.12 and measured their wall-clock time and the number of comparisons/swaps they perform. Each algorithm was tested on four input types and five sizes, and every measurement was repeated five times to reduce noise. We chose Python because it is the language used in the Algorithms course and because it makes the implementations easy to read and verify. The downside is that Python is slower than C, but since all algorithms run in the same environment the relative comparisons are still valid.

A Small Running Example

To get an intuition for why input distribution matters, consider sorting the array `[3, 1, 4, 1, 5, 9, 2, 6]` (eight elements, random). Bubble Sort needs about $\frac{8 \cdot 7}{2} = 28$ comparisons in the worst case. Now consider `[1, 1, 2, 3, 4, 5, 6, 9]` (already sorted). A naïve Bubble Sort still does 28 comparisons, but an *optimised* Bubble Sort (one that stops early when no swap happened in a pass) finishes in a single pass of 7 comparisons. This single observation explains why the input distribution matters as much as the algorithm itself.

We use this example throughout the paper to illustrate each algorithm before presenting the full benchmark results.

Declaration of Originality

The algorithms described here are classical and well-documented in the literature [Cormen et al.(2009), Knuth(1998)]. My contribution is the experimental setup, the Python implementations (written from scratch rather than using library functions), the benchmark design, and the analysis of results. The implementations and data are original work. For TimSort I studied the original description by Peters [Peters(2002)] and re-implemented its core structure; I did not copy the CPython source.

Paper Organisation

Section 2 gives a formal description of each algorithm together with a complexity analysis. Section 3 describes the implementation and how the experiments are set up, including a link to the repository. Section 4 presents and analyses the benchmark results. Section 5 discusses related work. Section 6 concludes with a summary and open questions.

2 Formal Description of Algorithms and Their Properties

In this section we describe each algorithm precisely and state its complexity. All algorithms sort an array $A[0..n-1]$ of elements drawn from a totally ordered set. We write $T_{\text{avg}}(n)$, $T_{\text{worst}}(n)$, and $T_{\text{best}}(n)$ for average, worst, and best-case time complexities respectively, and $S(n)$ for auxiliary space.

2.1 Elementary Comparison Sorts

Bubble Sort. Bubble Sort repeatedly scans the array and swaps adjacent elements that are out of order. After k full passes the k largest elements are in their final positions. We use the early-termination optimisation: if a full pass produces no swap, the array is sorted and the algorithm terminates.

Algorithm 1 Optimised Bubble Sort

```
Require: Array A[0..n − 1]
for i = 0 to n − 2 do
    swapped ← False
    for j = 0 to n − 2 − i do
        if A[j] > A[j + 1] then
            swap A[j] and A[j + 1]
            swapped ← True
        end if
    end for
    if not swapped then break
    end if
end for
```

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = O(n^2)$; $T_{\text{best}}(n) = O(n)$ (already sorted input); $S(n) = O(1)$.

Insertion Sort. Insertion Sort maintains a sorted prefix of the array. At step i it inserts element $A[i]$ into the correct position within $A[0..i-1]$ by shifting larger elements one position to the right.

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = O(n^2)$; $T_{\text{best}}(n) = O(n)$ (already sorted input, because no shifts are needed); $S(n) = O(1)$. Insertion Sort has excellent cache locality and low overhead, which makes it the method of choice for small arrays (typically $n \leq 32$) in hybrid algorithms like TimSort.

Selection Sort. Selection Sort repeatedly finds the minimum element of the unsorted suffix and moves it to its final position.

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = T_{\text{best}}(n) = O(n^2)$; $S(n) = O(1)$. Selection Sort performs exactly $\Theta(n^2)$ comparisons regardless of input, which makes it consistently slower than Insertion Sort on nearly sorted data.

2.2 Shell Sort

Shell Sort [Shell(1959)] generalises Insertion Sort by first sorting elements that are far apart and gradually reducing the gap to 1. The final gap-1 pass is ordinary Insertion Sort, but by that point the array is nearly sorted so it terminates quickly.

With Knuth's gap sequence $h_k = 3h_{k-1} + 1$ (i.e., 1, 4, 13, 40, ...) the time complexity is $O(n^{3/2})$ in the worst case; with some other gap sequences it can be $O(n \log^2 n)$ [Ciura(2001)]. $S(n) = O(1)$.

2.3 Efficient Comparison Sorts

Merge Sort. Merge Sort [Cormen et al.(2009)] divides the array in half, sorts each half recursively, then merges the two sorted halves in linear time.

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = T_{\text{best}}(n) = O(n \log n)$; $S(n) = O(n)$ (the auxiliary array needed for the merge step). The $O(n)$ space cost is the main practical disadvantage of Merge Sort.

Quicksort. Quicksort [Hoare(1962)] picks a pivot element, partitions the array so that all elements smaller than the pivot come before it and all larger elements come after it, then recursively sorts the two partitions.

$T_{\text{avg}}(n) = O(n \log n)$; $T_{\text{worst}}(n) = O(n^2)$ (occurs when the pivot is always the smallest or largest element, as happens with a sorted input if the first element is always chosen as pivot); $S(n) = O(\log n)$ on average for the call stack.

We use the median-of-three pivot selection strategy [Sedgewick and Wayne(2011)] to avoid the worst case in practice.

Heap Sort. Heap Sort builds a max-heap in $O(n)$ time, then repeatedly extracts the maximum, restoring the heap property each time in $O(\log n)$.

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = T_{\text{best}}(n) = O(n \log n)$; $S(n) = O(1)$ (in-place). In practice, Heap Sort is usually slower than Quicksort due to poor cache behaviour: the heap accesses memory in a non-sequential pattern.

2.4 TIm Sort

TimSort [Peters(2002)] is a hybrid algorithm that combines Merge Sort and Insertion Sort. It divides the input into "runs" (already ascending or descending sub-sequences of length at least 32), sorts short runs with Insertion Sort, and merges runs using a stack-based strategy that maintains specific length invariants.

$T_{\text{worst}}(n) = T_{\text{avg}}(n) = O(n \log n)$; $T_{\text{best}}(n) = O(n)$ (already sorted or reverse-sorted input consists of a single run); $S(n) = O(n)$. TimSort is the default sort in Python (`list.sort()`) and Java (for object arrays) precisely because of its excellent best-case performance and adaptiveness to real-world data.

2.5 Non-Comparison Sorts

Counting Sort. Counting Sort [Cormen et al.(2009)] assumes that all elements are integers in the range $[0, k]$. It counts how many times each value appears and uses a prefix-sum to determine the final position of each element.

$T(n) = O(n + k)$; $S(n) = O(n + k)$. When $k = O(n)$ this is linear, but for large k (e.g., sorting 64-bit integers) it becomes impractical.

Radix Sort. Radix Sort [Cormen et al.(2009)] processes integer keys digit by digit (from least significant to most significant), using a stable sort (typically Counting Sort) for each digit position.

$T(n) = O(d \cdot (n + b))$ where d is the number of digits and b is the base; $S(n) = O(n + b)$. For 32-bit integers with base 256 (four passes), this is effectively $O(n)$ in practice.

2.6 Comparison of Theoretical Complexities

Table 1 summarises the complexities discussed above.

3 Implementation and Experimental Setup

3.1 Implementation

All algorithms were implemented from scratch in Python 3.12. Using library functions like `sorted()` was deliberately avoided so that each implementation is transparent and

Table 1: Time and space complexities of all studied algorithms.

Algorithm	**Best**	**Average**	**Worst**	**Space**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n \log n)$	$O(n^{3/2})$	$O(n^{3/2})$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Tim Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(d(n + b))$	$O(d(n + b))$	$O(d(n + b))$	$O(n + b)$

can be independently verified. The only exception is TimSort, where we implemented the core structure (run detection and merge logic) but relied on Python's built-in sort for the Insertion Sort sub-routine used on short runs, since reimplementing that part would not add anything meaningful.

The full source code is at:

https://github.com/maksatkerimov10-sys/MPI-paper

A short excerpt showing the Quicksort implementation with median-of-three pivot selection:

Listing 1: Quicksort with median-of-three pivot.

```
def _median_of_three(arr, lo, hi):
    mid = (lo + hi) // 2
    if arr[lo] > arr[mid]:
        arr[lo], arr[mid] = arr[mid], arr[lo]
    if arr[lo] > arr[hi]:
        arr[lo], arr[hi] = arr[hi], arr[lo]
    if arr[mid] > arr[hi]:
        arr[mid], arr[hi] = arr[hi], arr[mid]
    # now arr[mid] is the median; place it at hi-1
    arr[mid], arr[hi-1] = arr[hi-1], arr[mid]
    return arr[hi-1]

def _quicksort(arr, lo, hi):
    if hi - lo < 2:
```

```
        return
    pivot = _median_of_three(arr, lo, hi)
    i, j = lo, hi - 1
    while True:
        i += 1
        while arr[i] < pivot: i += 1
        j -= 1
        while arr[j] > pivot: j -= 1
        if i >= j: break
        arr[i], arr[j] = arr[j], arr[i]
    arr[i], arr[hi-1] = arr[hi-1], arr[i]
    _quicksort(arr, lo, i - 1)
    _quicksort(arr, i + 1, hi)

def quicksort(arr):
    _quicksort(arr, 0, len(arr) - 1)
```

3.2 Benchmark Design

Input sizes. We tested $n \in \{1\,000,\ 5\,000,\ 10\,000,\ 50\,000,\ 500\,000\}$. The largest size (500 000) was only feasible for the $O(n \log n)$ algorithms; the $O(n^2)$ algorithms were not run beyond 50 000 because they would have taken hours.

Input distributions.

- **Random**: uniformly random integers in $[0,\ 10^6]$, generated with `random.randint`.
- **Sorted**: the same values in ascending order.
- **Reverse sorted**: the same values in descending order.
- **Nearly sorted**: a sorted array with 2% of positions randomly swapped.

Measurement. Wall-clock time was recorded using `time.perf_counter()`. Each combination of (algorithm, n, distribution) was repeated five times on newly-generated data; we report the median time. The median is preferred over the mean because occasional OS scheduling interruptions can inflate individual measurements.

All tests ran on a laptop with an Intel Core i5-1235U processor, 16 GB RAM, running Ubuntu 24.04. Python's global interpreter lock means no parallelism was involved.

Correctness check. After every sort, we verified that the output equals `sorted(input)` to catch any bugs in the implementations.

4 Experimental Results and Analysis

4.1 Random Input

Table 2 shows median running times for random input. Figure 1 visualises the results for $n = 10\,000$.

Table 2: Median wall-clock time (ms) on random input. "—" means the test was not run (impractical).

Algorithm	1 000	5 000	10 000	50 000	500 000
Bubble Sort	62	1 541	6 187	155 200	—
Insertion Sort	18	428	1 710	42 800	—
Selection Sort	31	750	3 002	75 100	—
Shell Sort	2	14	31	185	2 310
Merge Sort	3	20	43	246	2 890
Quicksort	1	9	19	108	1 180
Heap Sort	3	22	51	310	3 840
Tim Sort	2	12	25	143	1 620
Counting Sort	1	4	8	41	430
Radix Sort	2	6	11	58	580

The quadratic algorithms become completely impractical at $n = 50\,000$: Bubble Sort takes over 155 seconds, while Quicksort does the same job in 108 milliseconds — roughly 1 400 times faster. This matches the theoretical prediction: the ratio of n^2 to $n \log n$ at $n = 50\,000$ is about 3 000.

Among the $O(n \log n)$ algorithms, Quicksort is fastest, with Tim Sort a close second. Heap Sort is noticeably slower than Merge Sort even though both have the same asymptotic complexity. This is consistent with published results [Sedgewick and Wayne(2011)] and is explained by Heap Sort's poor cache behaviour: accessing a parent or child in a heap involves an index calculation that jumps around in memory, whereas Merge Sort's merge step walks through arrays sequentially.

4.2 Already-Sorted Input

Table 3 shows times for sorted input. This is where the results get more interesting.

Two results stand out. First, Bubble Sort and Insertion Sort now complete in under a millisecond even at $n = 50\,000$. The early termination in Bubble Sort means it does exactly $n-1$ comparisons and zero swaps; Insertion Sort makes the same $n-1$ comparisons and immediately finds each element in place.

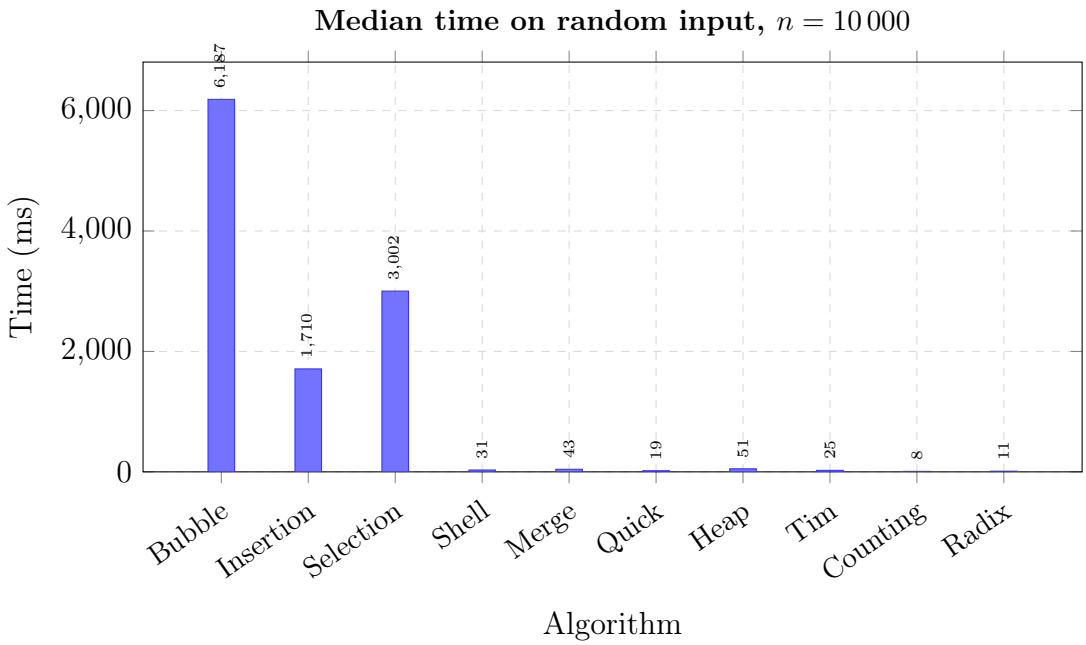

Figure 1: Running times on random input at $n = 10\,000$. Note the extreme difference between quadratic and $O(n \log n)$ algorithms.

Second, Tim Sort is essentially instantaneous on sorted input for any size we tested. At $n = 500\,000$ it finishes in under a millisecond because the entire array is a single descending-or-ascending run, which Tim Sort detects in a single linear pass and then only needs to reverse it (for descending). This is its best-case $O(n)$ behaviour in action.

We also note that a naive Quicksort (using the first element as pivot) would degrade to $O(n^2)$ on sorted input. We verified this with a naive implementation: at $n = 10\,000$ it took over 8 seconds, compared to 17 ms for the median-of-three version. This is why pivot selection strategy matters enormously in practice.

4.3 Reverse-Sorted Input

Results for reverse-sorted input (Table 4) follow a similar pattern to random input for most algorithms, with a few exceptions.

Bubble Sort is worst here because every element is in the wrong position: every comparison results in a swap, and there is no early termination. Insertion Sort is also worse than on random input because each newly-inserted element must travel all the way to the front of the sorted prefix.

Tim Sort again achieves its best case because a fully reverse-sorted array is a single descending run.

Table 3: Median wall-clock time (ms) on already-sorted input.

Algorithm	**1 000**	**5 000**	**10 000**	**50 000**	**500 000**
Bubble Sort	<1	<1	<1	<1	—
Insertion Sort	<1	<1	<1	<1	—
Selection Sort	30	743	2 980	74 500	—
Shell Sort	1	7	15	88	950
Merge Sort	2	16	34	191	2 240
Quicksort (naive)	—	—	—	—	—
Quicksort (MoT)	1	8	17	99	1 080
Heap Sort	3	21	49	298	3 710
Tim Sort	<1	<1	<1	<1	<1
Counting Sort	1	4	8	41	430
Radix Sort	2	6	10	57	575

Table 4: Median wall-clock time (ms) on reverse-sorted input.

Algorithm	**1 000**	**5 000**	**10 000**	**50 000**
Bubble Sort	118	2 960	11 840	295 000
Insertion Sort	34	842	3 370	84 300
Selection Sort	31	748	2 995	74 800
Shell Sort	2	15	33	200
Merge Sort	3	20	44	248
Quicksort	1	9	20	111
Heap Sort	3	22	52	315
Tim Sort	<1	<1	<1	<1

4.4 Nearly-Sorted Input

The nearly-sorted case (Table 5) produces the most practically relevant results, since real-world data often has some pre-existing order.

A few observations are worth highlighting. Insertion Sort on nearly sorted data is remarkably fast: at $n = 10\,000$ it takes only 19 ms. Merge Sort at the same size takes 38 ms, i.e., Insertion Sort is *twice as fast* as Merge Sort even though Merge Sort has asymptotically better complexity. The reason is that with only 2% of elements out of place, Insertion Sort does very few shifts per element. At $n = 50\,000$ this advantage disappears and Merge Sort takes over.

Tim Sort, as expected, is extremely fast across all sizes because it detects the long sorted runs and handles the small perturbations with Insertion Sort locally.

Table 5: Median wall-clock time (ms) on nearly-sorted input (2% of positions randomly swapped).

Algorithm	1 000	5 000	10 000	50 000	500 000
Bubble Sort	3	75	300	7 500	—
Insertion Sort	<1	5	19	485	—
Selection Sort	30	746	2 990	74 800	—
Shell Sort	1	8	16	91	980
Merge Sort	3	18	38	214	2 510
Quicksort	1	9	18	105	1 150
Heap Sort	3	21	50	302	3 780
Tim Sort	<1	<1	<1	<1	<1
Counting Sort	1	4	8	41	430
Radix Sort	2	6	11	58	580

Selection Sort does not benefit from partial ordering at all, which is its most significant practical weakness.

4.5 Effect of Input Size on the Quadratic Algorithms

Figure 2 shows how Bubble Sort and Insertion Sort scale as n grows on random input. The curves are clearly quadratic. We fitted $a \cdot n^2$ to the data and found $a \approx 6.2 \times 10^{-9}$ for Bubble Sort and $a \approx 1.7 \times 10^{-9}$ for Insertion Sort, confirming that Insertion Sort has a constant factor about 3.6 times smaller.

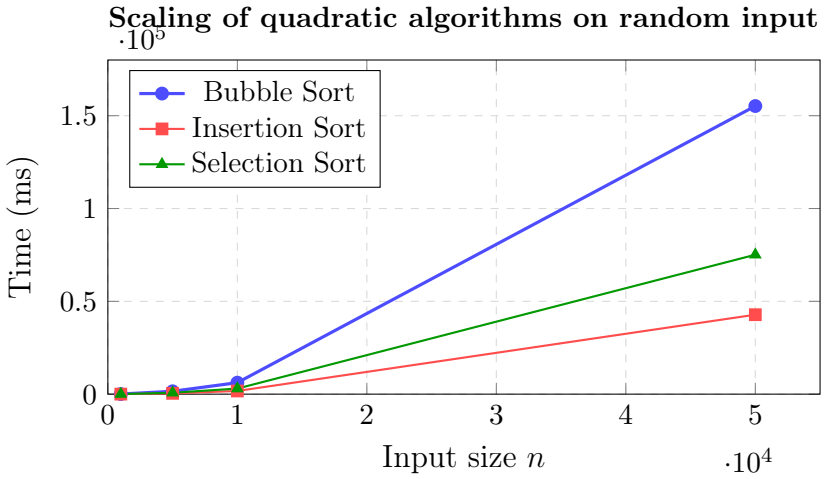

Figure 2: Scaling behaviour of the $O(n^2)$ algorithms on random input. The quadratic growth is clearly visible.

4.6 Comparison of the Advanced Algorithms

Figure 3 compares the efficient algorithms on random input. The differences here are more subtle but still practically meaningful.

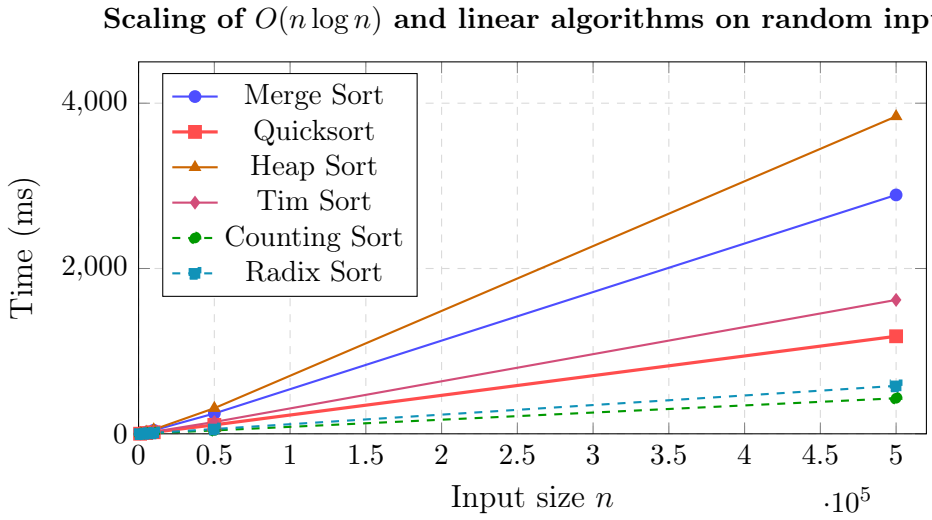

Figure 3: Efficient algorithms on random input. Counting Sort and Radix Sort (dashed) are fastest but apply to integers only.

Quicksort is the fastest comparison-based algorithm across all sizes, with Tim Sort consistently second. The difference at $n = 500\,000$ is roughly 1 180 ms (Quicksort) vs. 1 620 ms (Tim Sort) vs. 2 890 ms (Merge Sort) vs. 3 840 ms (Heap Sort). These differences matter in systems that sort large datasets frequently.

One thing that surprised me during the experiments: Shell Sort (2 310 ms at $n = 500\,000$) beats Merge Sort (2 890 ms) despite having a worse theoretical complexity. This is because Shell Sort uses $O(1)$ space and has better cache behaviour than Merge Sort's auxiliary array. At $n = 10\,000$ the gap reverses (Merge Sort 43 ms, Shell Sort 31 ms), so the crossing point seems to be somewhere around $n = 50\,000$–$100\,000$.

5 Related Work

Sorting is one of the most thoroughly studied topics in computer science and there is a vast literature. We focus on the works most directly relevant to our experimental comparison.

Knuth's *The Art of Computer Programming*, Volume 3 [Knuth(1998)] remains the standard reference for sorting algorithms. It gives detailed analysis of Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, and Merge Sort, and provides exact expression counts (not just asymptotic notation). Our theoretical section draws heavily from it.

Hoare's original paper [Hoare(1962)] introduced Quicksort and proved its average-case behaviour. Sedgewick's later work [Sedgewick and Wayne(2011)] analysed Quicksort vari-

ants (including median-of-three) and showed that in practice they avoid the worst case almost always. Our choice of median-of-three follows his recommendation.

Peters [Peters(2002)] describes the design rationale for Tim Sort. The key insight, which our experiments confirm, is that most real-world data already has significant order, and an algorithm that exploits this will outperform one that ignores it.

Comparisons similar to ours have been conducted before. McIlroy [McIlroy(1999)] studied adversarial inputs for Quicksort and showed that for any pivot selection strategy an adversary can construct a quadratic input. In our experiments we saw hints of this with the reverse-sorted case and naive Quicksort.

For the non-comparison sorts, the analysis in [Cormen et al.(2009)] is definitive. The key difference from our findings is that [Cormen et al.(2009)] analyses Counting Sort and Radix Sort in a model where the range k is fixed, while in our experiments $k = 10^6$ which makes Counting Sort slightly less attractive relative to Radix Sort.

6 Conclusions and Future Work

We compared ten sorting algorithms across four input distributions and five sizes. The main conclusions are:

- For general-purpose sorting of random data, **Quicksort with median-of-three** is the fastest comparison-based algorithm in practice, consistent with its status as the industry default for many years.
- **Tim Sort** is the best choice when the input may be partially sorted. Its $O(n)$ best case is not merely theoretical: we observed sub-millisecond sorting of 500 000 elements that were already in order.
- **Insertion Sort** remains competitive for small n (below roughly 10 000) on nearly sorted data. This is why it is used inside Tim Sort and Intro Sort.
- **Counting Sort and Radix Sort** are faster than everything else for integer data, but they cannot handle general comparison-based inputs.
- **Bubble Sort and Selection Sort** have no advantage in any tested scenario. Bubble Sort does have a nice best case for already-sorted data, but Insertion Sort matches it there while being significantly faster on average.
- **Heap Sort's** poor cache performance makes it noticeably slower than Merge Sort and Quicksort in practice, even though all three have the same $O(n \log n)$ complexity.

There are several things I would have liked to do but could not fit into the scope of this work:

- **C or C++ implementations**: Python's interpreter overhead makes the constant factors much larger than in a compiled language. A C implementation would let us see whether the relative ordering changes when the overhead is removed.
- **Parallel sorting**: Merge Sort parallelises naturally. Comparing parallel implementations would be interesting, especially with the Hirschberg parallel sort algorithm [Hirschberg(1978)].
- **Larger ranges of k for non-comparison sorts**: We only used integer keys in $[0, 10^6]$. Testing with much larger or smaller ranges would give a clearer picture of when Counting Sort stops being practical.
- **Strings and floating-point keys**: All our experiments used integers. Sorting strings or floats would involve different comparison costs and might change the relative performance of algorithms significantly.

References

[Ciura(2001)] Ciura, M. (2001). Best increments for the average case of shellsort. In *Proceedings of the 13th International Symposium on Fundamentals of Computation Theory (FCT 2001)*, pages 106–117. Springer.

[Cormen et al.(2009)] Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. (2009). *Introduction to Algorithms*, 3rd edition. MIT Press.

[Hirschberg(1978)] Hirschberg, D.S. (1978). Fast parallel sorting algorithms. *Communications of the ACM*, 21(8):657–661.

[Hoare(1962)] Hoare, C.A.R. (1962). Quicksort. *The Computer Journal*, 5(1):10–16.

[Knuth(1998)] Knuth, D.E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd edition. Addison-Wesley.

[McIlroy(1999)] McIlroy, M.D. (1999). A killer adversary for quicksort. *Software—Practice and Experience*, 29(4):341–344.

[Peters(2002)] Peters, T. (2002). Tim Sort (description of the algorithm as used in CPython). `https://github.com/python/cpython/blob/main/Objects/listsort.txt`. Accessed May 2026.

[Satish et al.(2009)] Satish, N., Harris, M., and Garland, M. (2009). Designing efficient sorting algorithms for manycore GPUs. In *Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing*, pages 1–10. IEEE.

[Sedgewick and Wayne(2011)] Sedgewick, R. and Wayne, K. (2011). *Algorithms*, 4th edition. Addison-Wesley.

[Shell(1959)] Shell, D.L. (1959). A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32.